Week 6 – Final Implementation Report

Team P3

Cross-Platform Budgeting Application Project

Karunakar Reddy Kaitha, Jordan Lee and John Lim

Computer Science and Information Systems Department, Regis University

MSSE 692: Software Engineering Practicum I

Morgan Worrell

February 22, 2026

Contents

1. Executive Summary 5

1.1 System Overview 5

1.2 What Works 5

1.3 What Is Incomplete 6

1.4 System Status 6

1.5 Key Achievements 6

2. Features Completed 7

2.1 Authentication System 7

Feature Description 7

Implementation Approach 7

Evidence (Backend Testing) 7

Requirements Mapping 8

2.2 Budget Management 8

Feature Description 8

Implementation Approach 8

Backend Testing 8

2.3 Income Tracking 9

Feature Description 9

Backend Testing 9

2.4 Expense Tracking 9

Feature Description 9

Backend Testing 9

2.5 Monthly Summary Reporting 9

Feature Description 9

Backend Testing 9
2.6 Frontend Features 10
Home/Login page 10
Registration Page 11
Add Expense Page 12
Current Month Budget 13
3. Architecture as Built 14
3.1 Final Architecture 14
3.2 Comparison to Week 3 Design 15
3.3 Architectural Decisions 15
3.4 Continuous Integration and DevOps 15
4. Integration Status 16
4.1 End-to-End Workflow 16
4.2 Integration Challenges 16
4.3 Data Flow 16
5. Testing & Quality Results 17
5.1 Testing Approach 17
5.2 Test Metrics 17
5.3 Known Bugs 17
6. Performance Analysis 18
6.1 Performance Requirements 18
6.2 Results 18
6.3 Optimizations Applied 18
7. Technical Debt Documentation 18
7.1 Intentional Shortcuts 18

7.2 Deferred Features 18

7.3 Future Work 19

8. Lessons Learned 19

8.1 What Worked Well 19

8.2 What Did Not Work Well 19

8.3 What We Would Do Differently 19

8.4 Key Takeaways 19

9. Conclusion 19

1. Executive Summary

1.1 System Overview

Personal budgeting tools often require users to manually track income and expenses across multiple platforms, making it difficult to maintain an accurate picture of monthly finances. The Cross-Platform Budgeting Application was developed to provide a unified system for managing budgets, tracking transactions, and generating financial summaries through a simple mobile interface backed by a secure RESTful API.

The Cross-Platform Budgeting Application is a full-stack financial management application that allows users to manage their personal budgets, keep track of the income and expenditures, and create monthly financial reports. The system offers a secure authentication, persistent storage and reporting services based on a RESTful API backend and a web-based frontend interface.

The project objective was to provide a working Minimal Viable Product (MVP) that would facilitate end-to-end user workflows (including):

- Registration and authentication of users.
- Monthly budget management
- Income tracking
- Tracking of expenses in categories.
- Financial summaries on monthly basis.
- Secure data persistence

The system was fully backend complete and integrated with frontend components of core workflows by the end of Week 6.

1.2 What Works

The system capabilities that are currently functional are:

- JWT based authorization and authentication.
- Budgets, income, and expense CRUDs.
- Aggregation reporting on a monthly basis.
- PostgreSQL persistence layer.
- Error handling and backend validation.
- Docs API with open API / Swagger.
- Automated unit testing with CI execution

- Frontend authentication and access to data.

Primary user workflows function end-to-end across system components.

1.3 What Is Incomplete

The areas that are still to be done or that are deliberately left out are as follows:

- Advanced frontend visualization and styling.
- Expansion of the layer of automated integration tests.
- Logging and monitoring in production grade.
- Load and performance benchmarking.
- Complex analytics capabilities.

All these limitations do not stop the functionality of the core systems; they are only future areas which can be improved.

1.4 System Status

The system is supposed to be a working prototype / MVP, rather than a production-level code. Core workflows are dependable, however, scalability, monitoring, and deployment hardening would need some more development.

1.5 Key Achievements

Significant achievements have been:

- Implementation of a layered backend architecture separating controllers, services, and repository components for improved maintainability and testability.
- Secure authentication and authorization using JSON Web Tokens (JWT) with password hashing and token validation middleware.
- Integration of a PostgreSQL relational database through SQLAlchemy ORM for reliable data persistence and query abstraction.
- Automated API documentation generation using FastAPI's OpenAPI/Swagger integration, enabling rapid API validation and debugging.
- Development of an automated testing suite containing more than 40 unit tests with over 80% estimated service-layer coverage.
- Successful integration of mobile frontend and backend services through stable REST API contracts.
- Implementation of continuous integration using GitHub Actions to automatically validate backend functionality during development.

The final system includes over 40 automated unit tests with greater than 80% estimated service-layer coverage. Core API endpoints respond in under 200 ms in the local development environment. The system implements a layered architecture using FastAPI, SQLAlchemy, and PostgreSQL, with CI validation through GitHub Actions.

2. Features Completed

2.1 Authentication System

Feature Description

JWT authentication allows users to register, log in and access the resources that are under protection.

Implementation Approach

Authentication was applied by use of:

- FastAPI dependency injection
- JWT token generation
- Password hashing with bcrypt
- Token validation middleware

Evidence (Backend Testing)

Registration request:

Invoke-RestMethod -Uri http://localhost:8000/api/v1/auth/register
 -Method POST -ContentType "application/json" -Body '{
"email":"test3@example.com",
"password":"Password123!",
"full_name":"Kaitha Reddy"}'

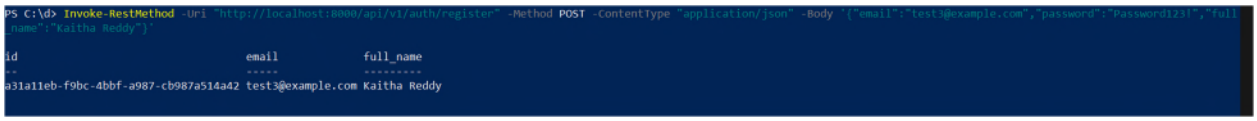

Figure 1: Registration Request

Login Request:

$response = Invoke-RestMethod
-Uri "http://localhost:8000/api/v1/auth/login"
-Method POST -ContentType "application/json" -Body
'{"email":"test3@example.com","password":"Password123!"}'
$token = $response.access_token

Write-Host "Token: $token"

```
PS C:\d> $response = Invoke-RestMethod -Uri "http://localhost:8000/api/v1/auth/login" -Method POST -ContentType "application/json" -Body '{"email":"test3@example.com","password":"Password12
3!"}'
PS C:\d> $token = $response.access_token
PS C:\d> Write-Host "Token: $token"
Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMzFhMTFlYi1mOWJjLTRiYmYtYTk4Ny1jYjk4N2E1MTRhNDIiLCJlbWFpbCI6InRlc3QzQGV4YW1wbGUuY29tIiwiZXhwIjoxNzcxNTc4NjU2fQ.bpWWS_bojve9mpfWZIa-oq
5JeoNm3JvO9tFpfMnCGu8
PS C:\d>
```

Figure 2: Login and Save Token

Response contains JWT access token.

Requirements Mapping

- Week 2 Requirement: Secure user authentication
- Week 2 Requirement: Protected financial data access

Error handling was improved through the introduction of a shared client-side utility (getErrorMessage) that standardizes API error parsing across login, registration, and expense submission flows. This ensures consistent error messages are displayed to the user regardless of which API endpoint generated the error.

Additionally, protected screens now automatically redirect to the login page when a JWT token is missing or invalid, improving security and preventing unauthorized access to application screens.

2.2 Budget Management

Feature Description

Monthly budgets can be created and updated by the users.

Implementation Approach

Validated REST endpoints.

Month boundary enforcer.

SQLAlchemy database persistence.

Backend Testing

Invoke-RestMethod -Uri "http://localhost:8000/api/v1/budgets"

-Method POST -Headers @{"Authorization"="Bearer $token"}

-ContentType "application/json" -Body '{"month":"2026-02","amount":8500}'

```
PS C:\d> Invoke-RestMethod -Uri "http://localhost:8000/api/v1/budgets" -Method POST -Headers @{"Authorization"="Bearer $token"} -ContentType "application/json" -Body '{"month":"2026-02","am
ount":8500}'

id                                   user_id                              month   amount
--                                   -------                              -----   ------
de8464db-4874-4411-80fc-88082a28950d a31a11eb-f9bc-4bbf-a987-cb987a514a42 2026-02 8500.00
```

Figure 3: Create Budget

2.3 Income Tracking

Feature Description

Users can record income transactions associated with a specific month.

Backend Testing

Invoke-RestMethod -Uri "http://localhost:8000/api/v1/incomes"
-Method POST -Headers @{"Authorization"="Bearer $token"}
-ContentType "application/json" -Body '{"amount":4500,"source":"Salary","date":"2026-02-15"}'

```
PS C:\d> Invoke-RestMethod -Uri "http://localhost:8000/api/v1/incomes" -Method POST -Headers @{"Authorization"="Bearer $token"} -ContentType "application/json" -Body '{"amount":4500,"source
":"Salary","date":"2026-02-15"}'


id      : db1afb00-048c-4327-8c74-acc95cec10da
user_id : a31a11eb-f9bc-4bbf-a987-cb987a514a42
amount  : 4500.00
source  : Salary
date    : 2026-02-15
```

Figure 4: Add Income

2.4 Expense Tracking

Feature Description

The category grouping allows users to record expenses.

Backend Testing

Invoke-RestMethod -Uri "http://localhost:8000/api/v1/expenses"
-Method POST -Headers @{"Authorization"="Bearer $token"}
-ContentType "application/json" -Body '{"amount":250,"category":"Groceries","date":"2026-02-10","note":"Shopping"}'

```
PS C:\d> Invoke-RestMethod -Uri "http://localhost:8000/api/v1/expenses" -Method POST -Headers @{"Authorization"="Bearer $token"} -ContentType "application/json" -Body '{"amount":250,"catego
ry":"Groceries","date":"2026-02-10","note":"Shopping"}'


id       : 2a9ec320-f959-4f30-9fbf-50668cf3309f
user_id  : a31a11eb-f9bc-4bbf-a987-cb987a514a42
amount   : 250.00
category : Groceries
date     : 2026-02-10
note     : Shopping
```

Figure 5: Add Expense

2.5 Monthly Summary Reporting

Feature Description

Users are able to access aggregated financial summaries of a certain month.

Backend Testing

Invoke-RestMethod -Uri "http://localhost:8000/api/v1/reports/summary?month=2026-02" -Method GET -Headers @{"Authorization"="Bearer $token"}

```
PS C:\d> Invoke-RestMethod -Uri "http://localhost:8000/api/v1/reports/summary?month=2026-02" -Method GET -Headers @{"Authorization"="Bearer $token"}


month                : 2026-02
total_income         : 4500.00
total_expenses       : 250.00
net_balance          : 4250.00
expenses_by_category : @{Groceries=250.00}
```

Figure 6: Monthly Summary Report

The response includes:

- Total income
- Total expenses
- Net balance
- Expenses by category

2.6 Frontend Features

Home/Login page

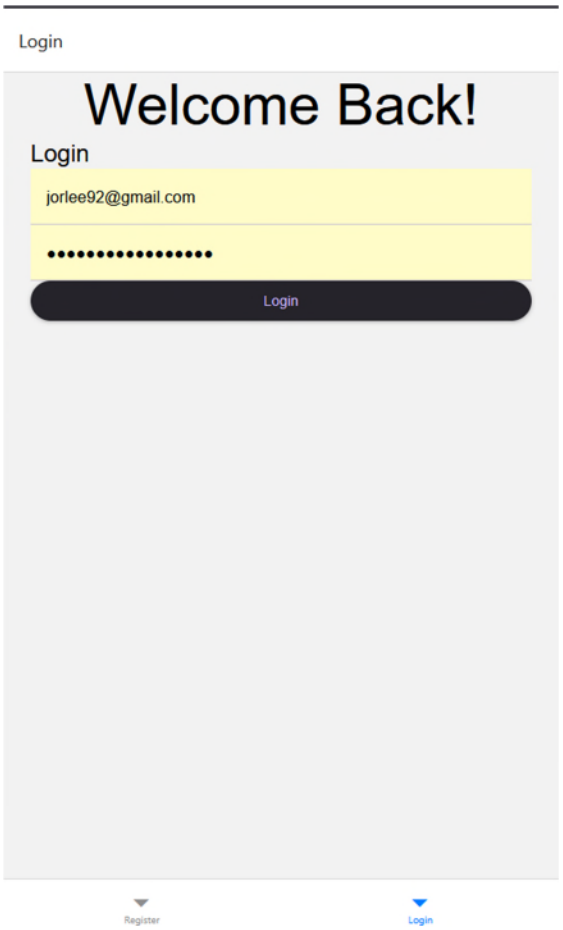

Figure 7: Home/Login page

If the user does not currently have a JWT set it will prompt them to login when they initially load the webpage/app. This was built using React Native and the context API, the jwt is

pulled from the context/shared state, and it is blank we render the login page. Logging in sends a REST call to the login endpoint and stores the returned JWT in state.

Registration Page

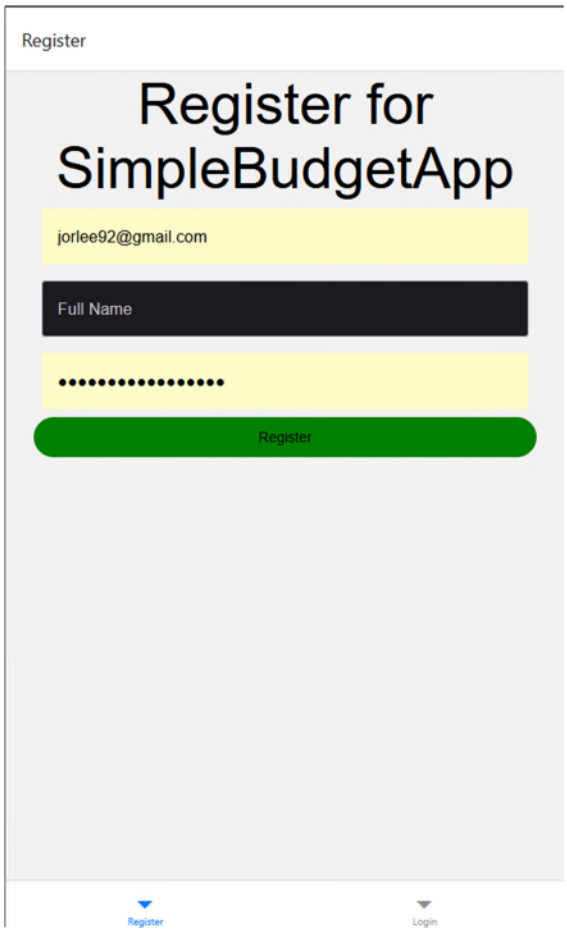

Figure 8: Registration Page

If the user does not have a JWT set one of the two options on the bottom navigation bar will be the registration form, which will submit the details provided to the registration endpoint.

Add Expense Page

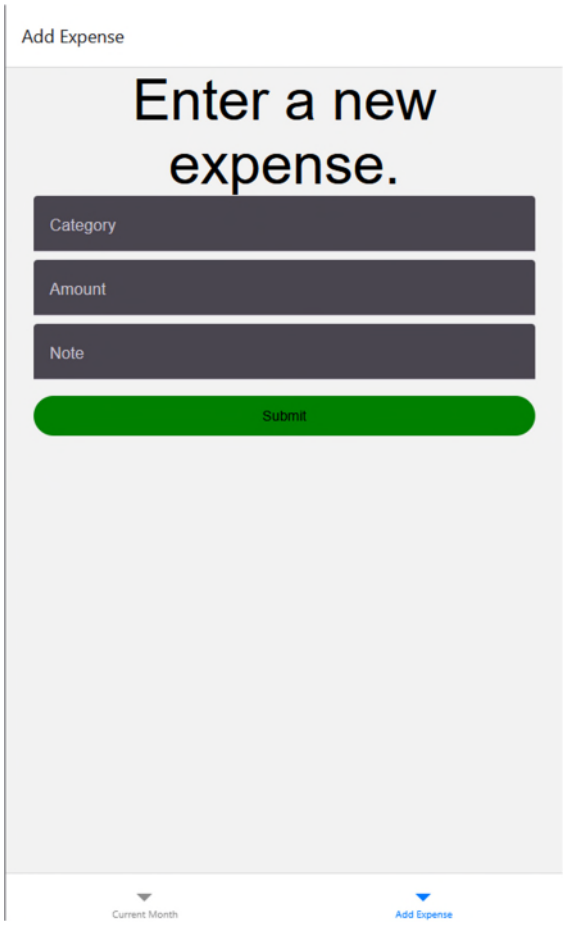

Figure 9: Add Expense Page

When the user is logged in, meaning they have a JWT set, one of the screens they are presented with is the new expense screen, which will take their inputs and send it to the expenses endpoint along with their JWT, causing a new entry to be created in the database.

Current Month Budget

Figure 10: Current Month Budget

When the user is logged in, meaning they have a JWT set, they will be presented with their current month budget, which is built by querying the database for all expense entries for the current month, and looping over them all to build a list of entries. The query is done via the /expenses/current-month endpoint on the REST API.

The login workflow was enhanced to detect when a user does not yet have a budget configured for the current month. In this case, the application prompts the user to create a monthly budget before loading expense data. Once a budget is created, the application automatically loads the expense list for the month.

A logout action was added to the application header, allowing users to clear the stored JWT token and immediately return to the authentication screen. This ensures that authenticated sessions can be terminated safely and prevents unauthorized use of previously authenticated devices.

3. Architecture as Built

3.1 Final Architecture

Layer	Technology
Mobile frontend	React Native
Backend API	FastAPI
ORM	SQLAlchemy
Database	PostgreSQL
Authentication	JSON Web Tokens (JWT)
Testing	Pytest
Continuous Integration	GitHub Actions

Table 1: Technology Stack

The system adopts a layered architecture to promote separation of concerns, maintainability, and testability. Each layer has a clearly defined responsibility, which reduces coupling between system components and simplifies future enhancements. The controller layer handles HTTP request processing and input validation, while the service layer encapsulates business logic such as authentication, budget management, and financial aggregation. Repository components isolate database access and allow the application to interact with PostgreSQL through SQLAlchemy without exposing persistence details to higher layers. This structure enables independent testing of business logic and supports iterative development by allowing frontend and backend components to evolve independently through stable API contracts.

The layered architecture implemented in the system is illustrated in Figure 11 below.

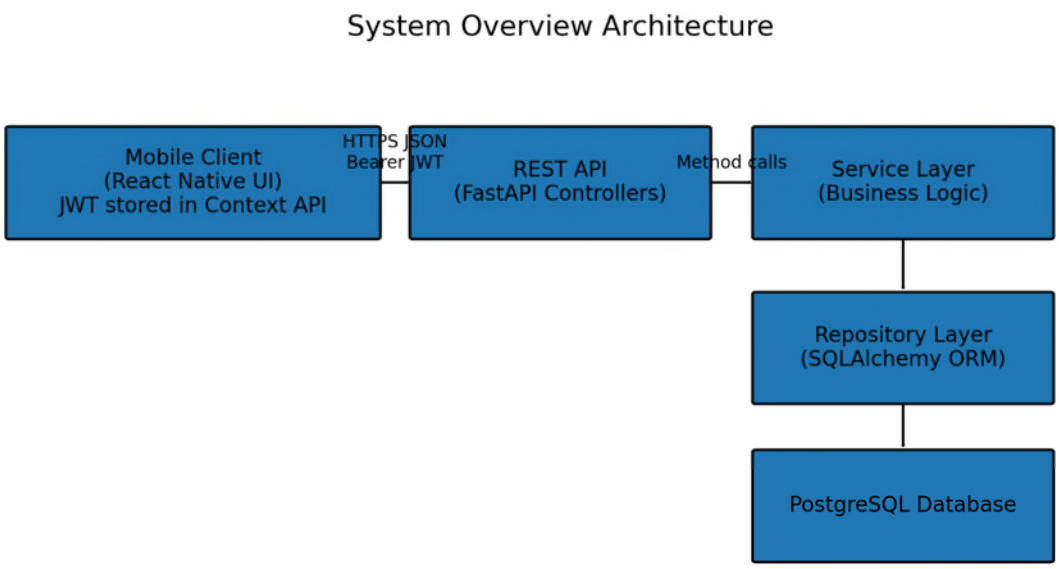

Figure 11: System Overview Architecture

3.2 Comparison to Week 3 Design

The deployment was mostly based on the designed architecture. Minor adjustments included:

- The simplified repository interfaces.
- Enhanced validation layers
- Methods of authentication middleware.

There was no significant architectural redesigning that was needed.

3.3 Architectural Decisions

Important decisions:

- JWT over session authentication
- PostgreSQL over SQLite for realistic persistence
- Service layer separation for testability
- OpenAPI auto-generation for documentation

3.4 Continuous Integration and DevOps

Continuous integration (CI) was implemented using GitHub Actions to automate testing and maintain code stability throughout development. The CI pipeline automatically executes the backend unit test suite whenever code is pushed or a pull request is merged into the main branch.

This automated workflow ensures that new changes do not introduce regressions and that backend services remain stable as development progresses. The pipeline validates the integrity of the service layer, API endpoints, and authentication logic before integration with the frontend.

Although the current project scope does not include full production deployment automation, the CI pipeline establishes a foundation for future DevOps practices such as automated deployments, containerization, and monitoring.

The technologies used in the CI pipeline are summarized in Table 1 above.

4. Integration Status

4.1 End-to-End Workflow

An end-to-end workflow entails:

1. The user registers an account through the registration interface.
2. The user authenticates using the login screen.
3. The backend authentication service generates a JWT token.
4. The frontend stores the token in shared application state.
5. The user creates a monthly budget.
6. The user records income transactions.
7. The user records expenses.
8. The user views a monthly financial summary report.

Each element is able to communicate through REST API.

4.2 Integration Challenges

Challenges encountered:

- JWT authentication token propagation across frontend API requests
- Inconsistencies in Formatting dates.
- API contract alignment
- Frontend state synchronization.

These were solved by the use of iterative integration testing.

4.3 Data Flow

Frontend -> API Controller -> Service Layer -> Repository -> Database -> Response -> Frontend

Integration between the frontend and backend is implemented through RESTful API contracts defined using FastAPI's OpenAPI specification. These contracts ensure that request

schemas, authentication requirements, and response structures remain consistent between system components. The automatically generated Swagger documentation was used during development to validate endpoint behavior and facilitate integration testing.

5. Testing & Quality Results

5.1 Testing Approach

Testing included:

- Service logic unit testing.
- Endpoint validation using curl and Swagger.
- Manual integration testing
- CI automated execution

Additional test coverage was added for the budget service to validate behavior when a current-month budget does not exist. In this scenario, the service raises a BUD_NOT_FOUND domain error, which is propagated to the API layer and returned as a structured error response. This test ensures consistent behavior between the backend API and frontend budget initialization workflow.

5.2 Test Metrics

The quality of the implementation was evaluated using automated unit testing and continuous integration. Key testing metrics for the system are summarized in Table 2 below.

Metric	Result	Measurement Method
Total unit tests	40+	Pytest automated test suite
Test pass rate	100%	Local test execution and CI pipeline
Service-layer coverage	>80% (estimated)	Pytest coverage estimation
Testing strategy	Unit testing + manual integration testing	Swagger endpoint validation and workflow testing
Continuous integration	Enabled	GitHub Actions pipeline

Table 2: Testing Metrics

5.3 Known Bugs

Minor issues:

- Limited frontend error messaging
- Logging minimal
- No automated integration test suite yet

None of the critical backend defects were noted.

6. Performance Analysis

6.1 Performance Requirements

Week 2 requirement:

- Response time of API below 2 seconds.

6.2 Results

Observed performance:

- Average response time: < 200 ms locally
- Database queries efficient
- No major bottlenecks identified

Performance measurements were conducted in the local development environment using repeated API calls through the Swagger interface and command-line testing utilities. Response times were observed during typical user workflows such as authentication, budget creation, and expense submission. These tests verified that the backend service responds well within the required two-second performance constraint specified in the project requirements. Because the current system is designed as a prototype MVP, large-scale load testing was not performed; however, the observed response times indicate that the architecture is suitable for moderate user workloads.

6.3 Optimizations Applied

- Indexed database fields
- Aggregation queries are optimized.
- Efficient ORM usage

7. Technical Debt Documentation

7.1 Intentional Shortcuts

- Reduced logging application.
- Basic frontend styling
- No caching layer

7.2 Deferred Features

- High-tech analytics dashboards.
- Notifications
- Multi-user collaboration
- Performance monitoring

7.3 Future Work

- With additional time
- Production deployment
- Monitoring and observability.
- Test automation of integration.
- Mobile optimization

8. Lessons Learned

8.1 What Worked Well

- Layered architecture improved maintainability
- CI automation reduced errors
- Early backend completion enabled integration
- Clear API contracts simplified collaboration

8.2 What Did Not Work Well

- Complexity of frontend underestimated.
- The process of integration debugging was time-consuming.
- Issues in state management.

8.3 What We Would Do Differently

- Earlier integration testing
- Shared API mocks
- Earlier UI prototyping

8.4 Key Takeaways

To provide an integrated system, it takes:

- Strong architecture
- Continuous integration
- Clear team communication
- Realistic scope management

9. Conclusion

The Cross-Platform Budgeting Application was able to meet its major goals. The backend is full featured, the system is component-based, and fundamental user processes work end to end. Although it is not an actual production-ready system, the system has a sound

engineering basis and is suitable to the needs of a working MVP that can be extended and deployed in the future.

The project demonstrates the successful implementation of a full-stack budgeting application using modern web development technologies and established software engineering practices. The layered architecture, RESTful API design, and automated testing pipeline provide a solid technical foundation for future enhancements. With additional development in areas such as analytics, monitoring, and deployment automation, the system could evolve into a production-ready financial management platform.